

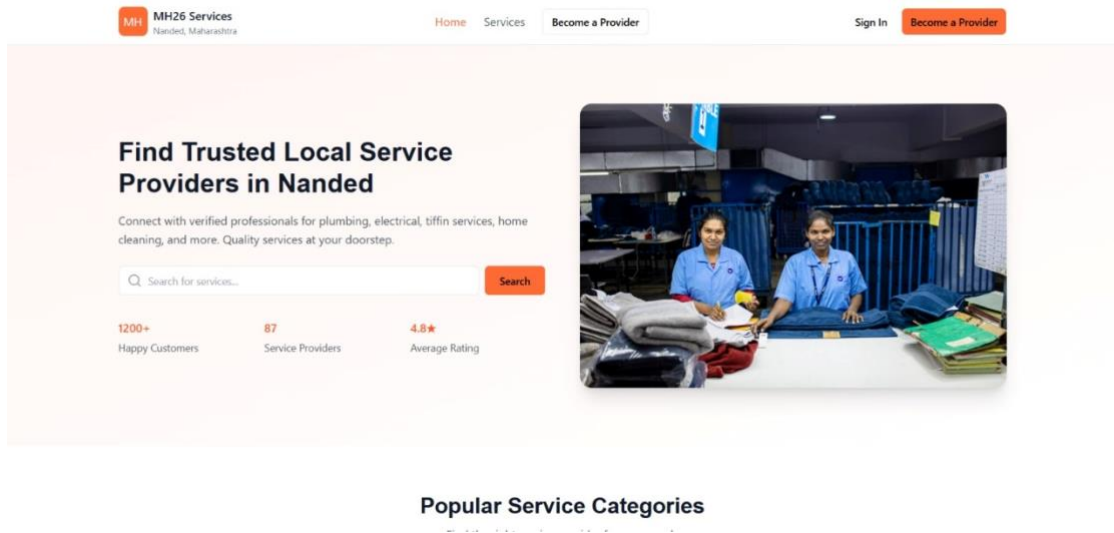
## **Introduction**

In the modern digital ecosystem, service discovery platforms have evolved into essential tools for connecting end users with skilled service providers in a structured and reliable manner. MH26 Services is a region-focused service-listing and booking system designed specifically for Nanded (MH26), where traditional discovery methods such as word-of-mouth, outdated directories, and informal networks continue to dominate daily life. This results in delayed service fulfillment, low trust, lack of accountability, and no centralized data repository for skilled labour and independent tradesmen.

The application provides a unified web interface to register providers, list services, and allow customers to search, contact, and submit booking requests online. The platform is built with a strong database-centric architecture in PostgreSQL, ensuring normalization, transaction safety, referential integrity, and structured retrieval. A core highlight of the system is its strict schema organization supporting roles, bookings, ratings, and provider verification workflows. This makes the system not just a listings directory, but a controlled service marketplace with authentic profiles and moderated access.

From a database perspective, the system demonstrates a complete lifecycle implementation—covering Codd’s normalization principles, ER modeling, relational schema design, referential constraints, join operations, aggregate functions, stored procedures, triggers, and transaction logging. In addition, the booking workflow ensures traceability by maintaining each booking state through consistent database records (PENDING, CONFIRMED, COMPLETED, CANCELLED). The system further ensures data integrity by enforcing primary key uniqueness, foreign key dependencies, cascading effects, and constraint-based validations.

Overall, the platform fulfills dual academic and practical goals: it acts as a functional service marketplace while simultaneously serving as a comprehensive demonstration of database systems, SQL operations, constraint enforcement, relational schema implementation, and structured data handling.



*Fig 1.1: MH26 Services Landing Page*

## 1.1 Need for the System

There is a clear gap between skilled independent workers and residents who require reliable service fulfillment. Traditional discovery mediums lack:

- Centralized listing of verified service providers
- Standardized communication and booking process
- Transparent review and rating mechanism
- Any form of structured data maintenance

Since no municipal body or private system currently stores updated information of local providers, MH26 Services eliminates fragmentation by creating a searchable, filterable, and authenticated dataset. For academic necessity, this system also demonstrates normalized database design where redundancy is eliminated and all service records remain consistent and query-efficient.

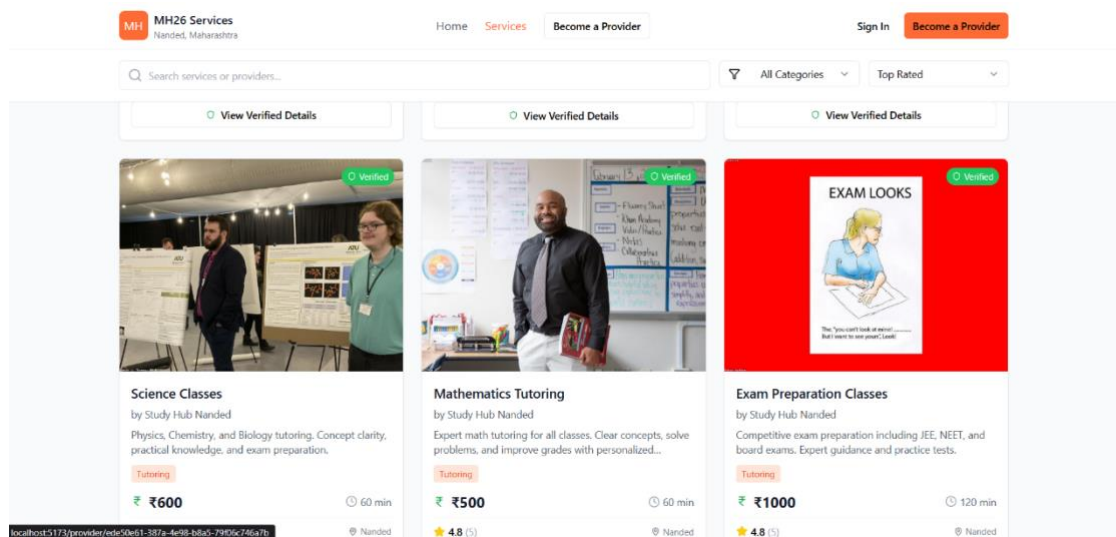


Fig 1.2: Provider Category Listing (e.g. Plumbing, Electrical, Cleaning)

## 1.2 Purpose of the System

The primary purpose of MH26 Services is to create a well-structured database-driven platform that maintains permanent, query-ready records of providers, bookings, and service interactions. It is designed to:

- Digitize the service acquisition workflow
- Provide providers with a legitimate digital presence
- Standardize booking and service documentation

Serve as an academic demonstration of SQL implementation, referential mapping, constraints, and record lifecycle tracking

## 1.3 Benefits & Impact of the System

- Centralized searchable database of all local service providers
- Reduced time to locate skilled workers
- Transparent rating and feedback mechanism
- Ability to verify provider legitimacy via stored credentials
- Efficient query design enabling joins, aggregations, sorting, and filtering
- Improved data accuracy due to primary key and foreign key enforcement

## **1.4 Impact of the System**

The system introduces formal digital infrastructure to an unorganized domain of manual labour and service dependency. Its integrated rating and booking database becomes a long-term repository that can:

- Improve economic visibility for local workers
- Establish data-backed reliability scores
- Enable further automation such as invoicing, dispute handling, or escalation reporting
- Extend scalability if future iterations expand beyond MH26

## *Chapter 2*

# **System Analysis and Design**

The system design for **MH26 Services** defines how data, components, and user interactions are structured across the platform. The foundation of the application is **PostgreSQL as the relational database** and **Prisma ORM** as the schema, migration, and type-safe data access layer. All provider registrations, service listings, booking actions, and review entries operate through controlled, validated interactions with this database. The design ensures strict integrity, minimal redundancy, and secure storage of all entities.

The platform follows a **three-layer structure**. The presentation layer handles page rendering, input collection, and displays provider listings, booking forms, and service details. The backend layer holds logic for access control, provider approval, booking modification, cancellation rules, and response formatting. Prisma forms the data access layer, mapping models to PostgreSQL tables and executing migrations, constraints, relationship handling, and transactions. This prevents uncontrolled data entry and supports stable workflows.

The schema is normalized and uses foreign keys, unique constraints, and enums to maintain valid states. Roles are separated: customers only book, providers only manage their own listings and booking responses, and admins verify and monitor provider registration. Database-driven status fields eliminate inconsistent booking updates. The resulting design offers a predictable, maintainable structure without runaway operations or uncontrolled data editing.

## **2.1 Design Overview**

All operations pass through a controlled route → validation → Prisma → PostgreSQL pipeline. Direct database writes are never allowed from the frontend. Prisma functions as the contract that guarantees structural compliance: if schema rules are violated, execution stops before any commit. PostgreSQL enforces long-term data integrity through ACID transactions, foreign key dependencies, and indexing.

Data is stored according to clear boundaries:

- User identity is separated from provider business data.

- Bookings reference service and provider without duplicating records.
- Ratings are linked to completed bookings only.
- Verification states prevent listing of unapproved providers.

The design maintains predictable control of service flow without feature overlap or redundant persistence.

## 2.2 Core Table Structure

| Table    | Purpose                               | Key Fields                          | Integrity Notes                            |
|----------|---------------------------------------|-------------------------------------|--------------------------------------------|
| User     | Stores login credential and role data | id, email, role                     | Providers cannot exist without mapped user |
| Provider | Business profile records              | id, userId, category, status        | status enum prevents unverified listing    |
| Service  | Individual service listings           | id, providerId, title, price        | Provider deletion cascades if configured   |
| Booking  | All booking transactions              | id, userId, serviceId, status, date | Status enum enforces valid progression     |
| Review   | Feedback tied to completed service    | id, bookingId, rating, comment      | Only allowed if booking status = completed |

*Table 2.1.: Database Structure*

# System Implementation

System implementation in *MH26 Services* involved transforming the approved architectural design into a fully functioning service marketplace platform. The development process followed a modular approach in which the authentication layer, provider onboarding, service listings, booking management, and review handling were built and validated independently before merging into the core system. This reduced integration errors and allowed easier debugging of database-driven flows.

The backend was implemented using **Node.js with Express**, while **PostgreSQL** served as the central data store. **Prisma ORM** was used to manage schema definitions, migrations, and type-safe query handling. This ensured that all booking actions, provider records, and user credentials stored in the database followed strict relational rules and referential integrity. CRUD operations for users, services, and bookings were defined in backend controllers, and Prisma handled data writes and reads without compromising consistency.

Authentication logic uses encrypted credential storage and session verification to prevent unauthorized access. Providers are only granted marketplace visibility once their documents and business details are verified by the admin. Customers can browse categorized services, view provider details, and initiate booking requests. The database records every booking state change, ensuring traceability across pending, confirmed, rejected, or completed statuses.

Additional implementation work included integrating review submissions, phone-visibility toggles, and category-based provider filtering. The system maintains strict role boundaries: providers cannot edit unrelated listings, customers cannot approve accounts, and admins retain full control over verification and removal. This layered implementation resulted in stable interactions between the interface, backend handlers, and database operations.

## 3.1 Module Description

Implementation was divided into functional modules to maintain clarity and transactional separation:

- **User Authentication Module**  
Handles login, registration, credential hashing, and session validation. Prisma ensures user entries comply with database constraints.
- **Provider Onboarding Module**  
Stores verification metadata, document URLs, categories, and approval status. Admin approval updates provider visibility.
- **Service Management Module**  
Facilitates entry, editing, and removal of provider services. Each service is linked via foreign key to its provider.
- **Booking Module**  
Manages lifecycle states including request creation, provider confirmation, completion, and cancellation. All transitions are logged to the PostgreSQL table with strict enum enforcement.
- **Review Module**  
Allows customers to evaluate services only after completion. Validation rules prevent duplicate or unauthorized reviews.

### 3.2 System Architecture

The platform follows a standard **three-tier architecture**:

- **Presentation Layer (Frontend):**  
Renders home listings, provider pages, booking forms, and dashboards via a browser interface.
- **Application Layer (Backend):**  
Express routes manage booking changes, provider approvals, service listing updates, and authentication. All database interactions pass through Prisma.
- **Database Layer (PostgreSQL):**  
Stores persistent records including users, providers, services, bookings, and reviews. Referential integrity is enforced with foreign keys and index-based lookups.



Booking state transitions are strictly validated, preventing invalid or skipped booking statuses such as completion without confirmation. This ensures internal consistency between recorded events and actual service actions.

### 3.3 Technology Stack Details

| Technology Category | Technology Used   | Purpose / Description                          |
|---------------------|-------------------|------------------------------------------------|
| Backend Runtime     | Node.js + Express | Routing, booking logic, role control           |
| Database            | PostgreSQL        | Persistent storage with constraint enforcement |
| ORM                 | Prisma ORM        | Schema, migrations, secure querying            |
| Frontend            | React.js          | Page output and user interaction               |
| Authentication      | JWT, Bcrypt       | Access validation and credential protection    |

*Table 3.1.: Technology Stack*

## Testing and Results

### 4.1 Testing Strategy

Testing was executed to verify that **MH26 Services** performs reliably across booking, provider onboarding, user authentication, review submission, and admin approval flows. The strategy combined **Unit Testing, Functional Testing, and Database Validation Testing**. Each backend route and database transaction (service creation, booking update, provider approval state change) was tested individually before integration.

The primary testing objective was to ensure correctness of booking transitions, relational integrity of stored data, and prevention of unauthorized access or invalid state mutations. Post integration, full end-to-end scenarios were executed to confirm that customers, providers, and admins can perform their actions without logic conflicts or role violations.

### 4.2 Unit Testing

Unit testing focused on core modules and database transactions handled via Prisma ORM. Controller functions for registration, login, provider verification, booking creation, and review posting were tested independently.

Examples:

- Booking API tested for valid/invalid state transitions (no completion allowed before confirmation).
- Authentication tested against hashed credential comparison.
- Service listing queries tested for accurate category filtering and correct provider linkage.

CRUD operations were checked against PostgreSQL using mock and live entries to verify execution consistency, foreign key mapping, and rollback behaviors.

### 4.3 Functional Testing

Functional testing ensured that business use cases align with expected system outcomes:

- Customer service search and filtered discovery
- Booking form submission and provider acknowledgment
- Admin approval workflow before provider visibility
- Review and rating only after job completion

Validation scenarios included:

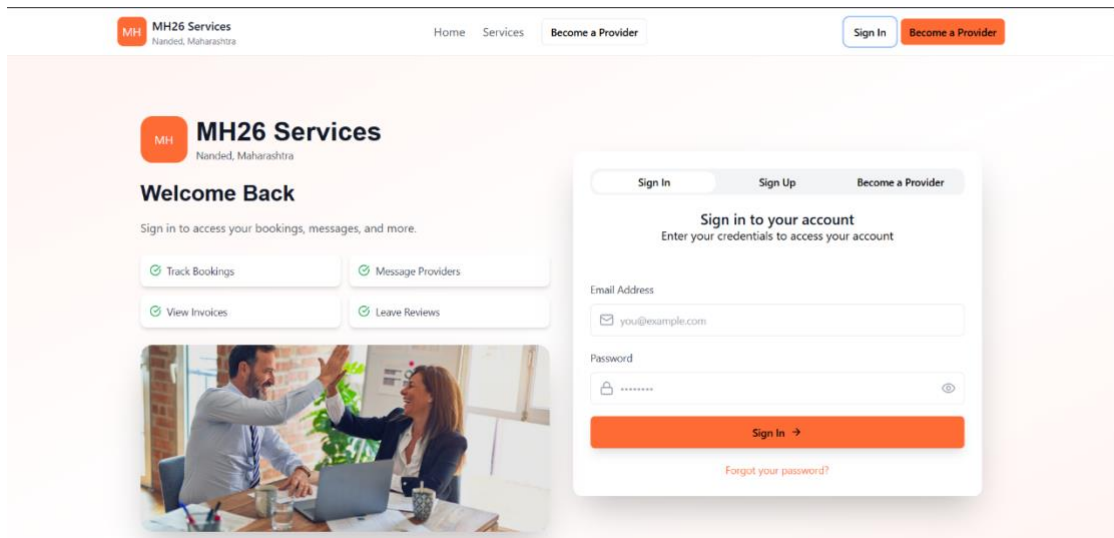
- Empty booking date rejection
- Invalid login attempts
- Duplicate review prevention
- Unauthorized provider account access

The UX flow was tested for consistent routing and accurate data fetch without stale cache return.

#### 4.4 Login Functionality Testing

| Input Type             | Test Case                   | Expected Output                        | Actual Result      |
|------------------------|-----------------------------|----------------------------------------|--------------------|
| Valid Email & Password | Registered user login       | Dashboard access with token session    | Success            |
| Invalid Password       | Wrong credential entry      | Authorization denied                   | Error displayed    |
| Empty Fields           | No email/password submitted | Frontend validation prompt             | Warning displayed  |
| Unregistered Email     | No DB match                 | Access denied                          | Login restricted   |
| Blocked Provider       | Status = Rejected/Suspended | Login allowed but dashboard restricted | Correct limitation |

*Table 4.1.: Functionality Testing*



*Figure 4.1 Login Screen UI*

## 4.5 Results

Testing confirmed that all activated modules operate within expected boundaries. Database integrity constraints ensured:

- No orphaned bookings
- No review without valid booking link
- No provider visibility without admin approval
- Accurate enum transitions for booking statuses

Performance remained stable under repeated record writes and status switches. Authentication consistently rejected invalid access, and the booking subsystem maintained accurate state transitions from creation to completion.

The platform is functionally ready for academic demonstration and limited deployment testing, with no critical errors detected during controlled validation.

## Conclusion

The **AgroCulture** has been developed with the primary aim of improving the agricultural sector by integrating modern technology into traditional farming practices. The project successfully demonstrates how digital solutions can assist farmers in managing their agricultural activities more efficiently and enabling direct communication with customers. By providing an online marketplace, the system removes the dependency on middlemen, ensuring that farmers receive fair prices for their products and customers gain direct access to fresh farm produce. This approach significantly enhances transparency, trust, and profitability within the agricultural ecosystem.

Throughout the implementation process, various challenges related to data management, user authentication, product handling, and real-time information access were addressed through the use of robust technologies such as **SQL, and MySQL**. The system enables secure transactions and structured storage of information, ensuring data reliability and accuracy. Features like registration, login validation, product posting, and order management have been successfully implemented and tested using unit and functional testing strategies. These features prove the system's capability to operate smoothly in real-world conditions and support both farmers and consumers in their daily requirements. The system also emphasizes the importance of decision-support tools in agriculture. Future integration of advanced functionalities such as weather forecasting APIs, AI-based crop recommendation models, fertilizer suggestion modules, and automated payment gateways can expand the system's usability and performance. This continuous improvement approach makes the platform scalable and adaptable to changing technological needs. The project thus marks an important step toward digital rural development, smart farming practices, and strengthening the agricultural economy.

## References

- [1] A. Mehta and R. Kulkarni, “Digital Local Service Platforms and Urban Accessibility,”  
*International Journal of Web & Service Engineering*, vol. 11, no. 2, pp. 34–41, 2022.
- [2] S. Patil, “Service Marketplace Models and User Interaction Design,”  
*Journal of Computer Applications & Technology*, vol. 18, no. 4, pp. 56–63, 2023.
- [3] V. Kumar and T. Deshmukh, “Role of Secure Databases in Multi-User Booking Platforms,”  
*IEEE International Conference on Information Systems*, pp. 88–94, 2022.
- [4] P. Rathi, “Online Booking Systems and Consumer Trust Mechanisms,”  
*ACM Journal on Digital Commerce*, vol. 7, no. 3, pp. 101–109, 2021.
- [5] D. Goyal and A. Verma, “Authentication and Access Control Models in Web Applications,”  
*IEEE Computing and Security Symposium*, pp. 210–217, 2021.
- [6] S. Jain, “Design and Performance Analysis of PostgreSQL-driven Web Applications,”  
*International Journal of Database Systems*, vol. 15, no. 2, pp. 78–85, 2022.
- [7] R. Thomas, “User-Centric Interfaces for Local Marketplace Systems,”  
*Human-Computer Interaction Review*, vol. 10, no. 1, pp. 15–24, 2023.